COMPUTATIONAL METHOD

LAB FILE



MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY

C4 , Janakpuri, New Delhi – 110058

Electronics and Communication Engineering Department

Name:- Rishi Parkash Verma

Class Serial No.:- 24

Enrollment No.:- 03315002825

Class:- ECE — 1

Submitted To:

Dr. Geetanjali

Dr. Deeba Naqvi

INDEX

Exp no.	Date	Practical name	Page no.	Teacher's sign	Remarks

Practical : 1 (a)

Linear equation in 1 variable

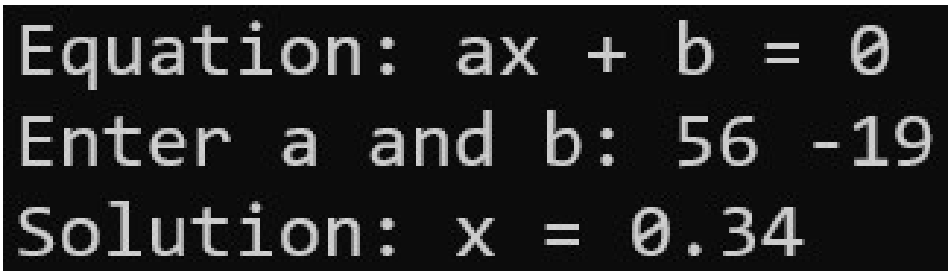
Aim : Program to find value of x in Linear equation.

Code :

```
#include <stdio.h>

int main() {
  float a, b, x;
  printf("Equation: ax + b = 0\n");
  printf("Enter a and b: ");
  scanf("%f %f", &a, &b);
  if (a == 0) {
    printf("Not a linear equation (a=0).\n");
  } else {
    x = -b / a;
    printf("Solution: x = %.2f\n", x);
  }
  return 0;
}
```

Output:



Practical : 1 (b)

Linear equation in 2 variable

Aim : Program to find value of x,y in Linear equation.

Code:

```
#include <stdio.h>

int main() {
  float a1, b1, c1, a2, b2, c2, x, y, det;
  printf("Write two Equations in ax + by = c form\n");
  printf("Enter a1, b1, c1: ");
  scanf("%f %f %f", &a1, &b1, &c1);
  printf("Enter a2, b2, c2: ");
  scanf("%f %f %f", &a2, &b2, &c2);
  printf("Equations are:\n");
  printf("%.2fx %c %.2fy = %.2f\n", a1, b1 >= 0 ? '+' : '-', b1 >= 0 ? b1 :
  b1, c1);
  printf("%.2fx %c %.2fy = %.2f\n", a2, b2 >= 0 ? '+' : '-', b2 >= 0 ? b2 :
  b2, c2);
  //cramer's rule
  det = a1 * b2 - a2 * b1;
  if (det == 0) {
    printf("No unique solution exists.\n");
  } else {
    x = (c1 * b2 - c2 * b1) / det;
    y = (a1 * c2 - a2 * c1) / det;
    printf("Solution: x = %.2f, y = %.2f\n", x, y);
  }
  return 0;
}
```

Output:

```
Write two Equations in ax + by = c form
Enter a1, b1, c1: 1 -2 -3
Enter a2, b2, c2: 2 4 -5
Equations are:
1.00x - 2.00y = -3.00
2.00x + 4.00y = -5.00
Solution: x = -2.75, y = 0.12
```

Practical : 1 (c)

Quadratic Equation

Aim : Program to find value of x in Quadratic equation.

Code:

```
#include <stdio.h>
#include <math.h>

int main() {
  float a, b, c, Det, r1, r2, realPart, imagPart;
  printf("Equation: ax^2 + bx + c = 0\n");
  printf("Enter a, b, c: ");
  scanf("%f %f %f", &a, &b, &c);
  printf("%.2fx^2 %c %.2fx %c %.2f = 0\n", a,
    b >= 0 ? '+' : '-', b >= 0 ? b : -b,
    c >= 0 ? '+' : '-', c >= 0 ? c : -c);

  Det = b * b - 4 * a * c;

  if (Det > 0) {
    r1 = (-b + sqrt(Det)) / (2 * a);
    r2 = (-b - sqrt(Det)) / (2 * a);
    printf("Roots are real and distinct: %.2f and %.2f\n", r1, r2);
  } else if (Det == 0) {
    r1 = -b / (2 * a);
    printf("Roots are real and equal: %.2f and %.2f\n", r1, r1);
  } else {
    realPart = -b / (2 * a);
    if (fabs(realPart) < 0.001)
      realPart = 0;
    imagPart = fabs(sqrt(-Det) / (2 * a));
    printf("Roots are complex: %.2f + %.2fi and %.2f - %.2fi\n", realPart,
    imagPart, realPart, imagPart);
  }
  return 0;
}
```

Output:

```
Equation: ax^2 + bx + c = 0
Enter a, b, c: 1 -5 6
1.00x^2 - 5.00x + 6.00 = 0
Roots are real and distinct: 3.00 and 2.00
```

```
Equation: ax^2 + bx + c = 0
Enter a, b, c: -1 0 -1
-1.00x^2 + 0.00x - 1.00 = 0
Roots are complex: 0.00 + 1.00i and 0.00 - 1.00i
```

Practical : 2

Bisection Method

Aim : Program for finding roots of f(x)=0 Bisection method.

Code:

```
#include <math.h>
#include <stdio.h>

float A, B, C, D;

float f(float x) {
  return ((A * x + B) * x + C) * x + D;
}

int main() {
  float a, b, mid, root1, e = 0.000001;
  float p, q, r, d, root2, root3;
  int iter = 0;

  printf("Cubic equation: Ax^3 + Bx^2 + Cx + D = 0\n");
  printf("Enter A, B, C, D: ");
  if (scanf("%f %f %f %f", &A, &B, &C, &D) != 4 || A == 0) {
    printf("Invalid cubic coefficients.\n");
    return 1;
  }

  printf("Enter interval for first root: ");
  if (scanf("%f %f", &a, &b) != 2 || f(a) * f(b) > 0) {
    printf("Invalid interval\n");
    return 1;
  }

  do {
    mid = (a + b) / 2;
    iter++;
    if (f(a) * f(mid) < 0)
      b = mid;
    else
      a = mid;
  } while (fabs(f(mid)) > e);

  root1 = mid;
  printf("Iterations took %d \n", iter);
  printf("First root (bisection): %.6f\n", root1);

  p = A;
  q = B + A * root1;
  r = C + q * root1;
  d = q * q - 4 * p * r;
```

```
  if (d >= 0) {
    root2 = (-q + sqrt(d)) / (2 * p);
    root3 = (-q - sqrt(d)) / (2 * p);
    printf("Second root: %.6f\n", root2);
    printf("Third root: %.6f\n", root3);
  } else {
    float real = -q / (2 * p);
    float imag = sqrt(-d) / fabs(2 * p);
    printf("Second root: %.6f + %.6fi\n", real, imag);
    printf("Third root: %.6f - %.6fi\n", real, imag);
  }

  return 0;
}
```

Output:

```
Cubic equation: Ax^3 + Bx^2 + Cx + D = 0
Enter A, B, C, D: 1 2 3 4
Enter interval for first root: -10 10
Iterations took 21
First root (bisection): -1.650629
Second root: -0.174685 + 1.546869i
Third root: -0.174685 - 1.546869i
```

Practical : 3

Secant Method

Aim : Program for finding roots of f(x)=0 Secant method.

Code:

```
#include <math.h>
#include <stdio.h>

float A, B, C, D;

float f(float x) {
  return ((A * x + B) * x + C) * x + D;
}

int main() {
  float x0, x1, x2, e = 0.000001;
  float p, q, r, d, root2, root3;
  int iter = 0;

  printf("Cubic equation: Ax^3 + Bx^2 + Cx + D = 0\n");
  printf("Enter A, B, C, D: ");
  if (scanf("%f %f %f %f", &A, &B, &C, &D) != 4 || A == 0) {
    printf("Invalid cubic coefficients.\n");
    return 1;
  }

  printf("Enter initial guesses: ");
  if (scanf("%f %f", &x0, &x1) != 2 || (f(x0) - f(x1)) == 0) {
    printf("Invalid initial guesses.\n");
    return 1;
  }

  do {
    x2 = (f(x1) * x0 - f(x0) * x1) / (f(x1) - f(x0));
    iter++;
    x0 = x1;
    x1 = x2;
  } while (fabs(f(x2)) > e);
  printf("Iterations took %d \n", iter);
  printf("First Root is : %f\n", x2);

  p = A;
  q = B + A * x2;
  r = C + q * x2;
  d = q * q - 4 * p * r;

  if (d >= 0) {
    root2 = (-q + sqrt(d)) / (2 * p);
    root3 = (-q - sqrt(d)) / (2 * p);
```

```
    printf("Second root: %.6f\n", root2);
    printf("Third root: %.6f\n", root3);
  } else {
    float real = -q / (2 * p);
    if (fabs(real) < 0.000001)
      real = 0;
    float imag = sqrt(-d) / fabs(2 * p);
    printf("Second root: %.6f + %.6fi\n", real, imag);
    printf("Third root: %.6f - %.6fi\n", real, imag);
  }

  return 0;
}
```

Output:

```
Cubic equation: Ax^3 + Bx^2 + Cx + D = 0
Enter A, B, C, D: 2 -3 4 -5
Enter initial guesses: -10 10
Iterations took 6
First Root is : 1.371134
Second root: 0.064433 + 1.348761i
Third root: 0.064433 - 1.348761i
```

Practical : 4

Newton Raphson Method

Aim: Program for finding roots of f(x)=0 Newton Raphson method.

Code:

```
#include <math.h>
#include <stdio.h>

float A, B, C, D;

float f(float x) {
  return ((A * x + B) * x + C) * x + D;
}

float df(float x) {
  return (3 * A * x * x) + (2 * B * x) + C;
}

int main() {
  float x0, x1, e = 0.000001;
  float p, q, r, d, root2, root3;
  int iter = 0;

  printf("Newton-Raphson Method for Ax^3 + Bx^2 + Cx + D = 0\n");
  printf("Enter A, B, C, D: ");
  if (scanf("%f %f %f %f", &A, &B, &C, &D) != 4 || A == 0) {
    printf("Invalid cubic coefficients.\n");
    return 1;
  }

  printf("Enter initial guess: ");
  if (scanf("%f", &x0) != 1) {
    printf("Invalid initial guess.\n");
    return 1;
  }

  do {
    if (df(x0) == 0) {
      printf("Derivative is zero. Cannot continue.\n");
      return 1;
    }
    x1 = x0 - f(x0) / df(x0);
    iter++;
    x0 = x1;
  } while (fabs(f(x1)) > e);

  printf("Iterations took %d\n", iter);
  printf("First root: %.6f\n", x1);
```

```
  p = A;
  q = B + A * x1;
  r = C + q * x1;
  d = q * q - 4 * p * r;

  if (d >= 0) {
    root2 = (-q + sqrt(d)) / (2 * p);
    root3 = (-q - sqrt(d)) / (2 * p);
    printf("Second root: %.6f\n", root2);
    printf("Third root: %.6f\n", root3);
  } else {
    float real = -q / (2 * p);
    if (fabs(real) < e)
      real = 0;
    float imag = sqrt(-d) / fabs(2 * p);
    printf("Second root: %.6f + %.6fi\n", real, imag);
    printf("Third root: %.6f - %.6fi\n", real, imag);
  }
  return 0;
}
```

Output:

```
Newton-Raphson Method for Ax^3 + Bx^2 + Cx + D = 0
Enter A, B, C, D: 2 -3 4 -5
Enter initial guess: 1
Iterations took 4
First root: 1.371134
Second root: 0.064433 + 1.348761i
Third root: 0.064433 - 1.348761i
```

Practical : 5

Matrix Operations

Aim: Program to perform basic Matrix Operations.

- Addition
- Subtraction
- Multiplication
- Inverse
- Adjoint
- Determinant
- Eigenvalues
- Simultaneous Equations
- Orthogonal Check

Code:

```
#include <math.h>
#include <stdio.h>
#define Max 10

float determinant_calc(float a[Max][Max], int n) {
  if (n == 1) return a[0][0];
  if (n == 2) return a[0][0] * a[1][1] - a[0][1] * a[1][0];
  float det = 0, sub[Max][Max];
  int sign = 1;
  for (int x = 0; x < n; x++) {
    int subi = 0;
    for (int i = 1; i < n; i++) {
      int subj = 0;
      for (int j = 0; j < n; j++) {
        if (j == x) continue;
        sub[subi][subj++] = a[i][j];
      }
      subi++;
    }
    det += sign * a[0][x] * determinant_calc(sub, n - 1);
    sign = -sign;
  }
  return det;
}

void getCofactor(float a[Max][Max], float temp[Max][Max], int p, int q, int n) {
  int i = 0, j = 0;
  for (int row = 0; row < n; row++) {
    for (int col = 0; col < n; col++) {
      if (row == p || col == q) continue;
      temp[i][j++] = a[row][col];
      if (j == n - 1) { j = 0; i++; }
```

```
    }
  }
}

void adjoint_calc(float a[Max][Max], float adj[Max][Max], int n) {
  if (n == 1) { adj[0][0] = 1; return; }
  int sign = 1; float temp[Max][Max];
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      getCofactor(a, temp, i, j, n);
      sign = ((i + j) % 2 == 0) ? 1 : -1;
      adj[j][i] = (sign) * (determinant_calc(temp, n - 1));
    }
  } }
void input_matrix(float a[Max][Max], int* r, int* c, int is_square) {
  printf("Enter rows and columns: ");
  if (is_square) {
    if (scanf("%d", r) != 1) return;
    *c = *r;
  } else {
    if (scanf("%d %d", r, c) != 2) return;
  }
  printf("Enter elements: ");
  for (int i = 0; i < *r; i++)
    for (int j = 0; j < *c; j++)
      if (scanf("%f", &a[i][j]) != 1) return;
}
void addition() {
  float a[Max][Max], b[Max][Max]; int r, c;
  printf("Enter rows and columns: ");
  if (scanf("%d %d", &r, &c) != 2) return;
  printf("Enter first matrix\nEnter elements: ");
  for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
      if (scanf("%f", &a[i][j]) != 1) return;
  printf("Enter second matrix\nEnter elements: ");
  for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
      if (scanf("%f", &b[i][j]) != 1) return;
  printf("Sum:\n");
  for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) printf("%.2f ", a[i][j] + b[i][j]);
    printf("\n");
  }
}

void subtraction() {
  float a[Max][Max], b[Max][Max]; int r, c;
  printf("Enter rows and columns: ");
  if (scanf("%d %d", &r, &c) != 2) return;
  printf("Enter first matrix\nEnter elements: ");
  for (int i = 0; i < r; i++)
```

```
    for (int j = 0; j < c; j++)
      if (scanf("%f", &a[i][j]) != 1) return;
  printf("Enter second matrix\nEnter elements: ");
  for (int i = 0; i < r; i++)
    for (int j = 0; j < c; j++)
      if (scanf("%f", &b[i][j]) != 1) return;
  printf("Difference:\n");
  for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) printf("%.2f ", a[i][j] - b[i][j]);
    printf("\n");
  }
}

void multiplication() {
  float a[Max][Max], b[Max][Max], mul[Max][Max] = { 0 }; int r1, c1, r2, c2;
  printf("Enter first matrix\n");
  input_matrix(a, &r1, &c1, 0);
  printf("Enter second matrix\n");
  input_matrix(b, &r2, &c2, 0);
  if (c1 != r2) {
    printf("Cannot multiply.\n");
    return;
  }
  printf("Product:\n");
  for (int i = 0; i < r1; i++) {
    for (int j = 0; j < c2; j++) {
      for (int k = 0; k < c1; k++) mul[i][j] += a[i][k] * b[k][j];
      printf("%.2f ", mul[i][j]);
    }
    printf("\n");
  }
}

void inverse() {
  float a[Max][Max], adj[Max][Max]; int n;
  printf("Enter the matrix\n");
  input_matrix(a, &n, &n, 1);
  float det = determinant_calc(a, n);
  if (det == 0) printf("Inverse does not exist.\n");
  else {
    adjoint_calc(a, adj, n); printf("Inverse:\n");
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) printf("%.4f ", adj[i][j] / det);
      printf("\n");
    }
  }
}

void adjoint_op() {
  float a[Max][Max], adj[Max][Max]; int n;
  printf("Enter the matrix\n");
  input_matrix(a, &n, &n, 1); adjoint_calc(a, adj, n);
```

```
  printf("Adjoint:\n");
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) printf("%.4f ", adj[i][j]);
    printf("\n");
  }
}

void determinant() {
  float a[Max][Max]; int n;
  printf("Enter the matrix\n");
  input_matrix(a, &n, &n, 1);
  printf("Determinant = %.4f\n", determinant_calc(a, n));
}

void eigenvalue() {
  float a[Max][Max], x[Max], nx[Max]; int n;
  printf("Enter the matrix\n");
  input_matrix(a, &n, &n, 1);
  printf("Enter initial guess vector: ");
  for (int i = 0; i < n; i++)
    if (scanf("%f", &x[i]) != 1)
      return;
  float lambda = 0, lambda_old = 0, max_val; int iter = 0;
  do {
    lambda_old = lambda;
    for (int i = 0; i < n; i++) {
      nx[i] = 0; for (int j = 0; j < n; j++) nx[i] += a[i][j] * x[j];
    }
    max_val = fabs(nx[0]);
    for (int i = 1; i < n; i++) if (fabs(nx[i]) > max_val) max_val = fabs(nx[i]);
    lambda = max_val;
    for (int i = 0; i < n; i++) x[i] = nx[i] / lambda;
    iter++;
  } while (fabs(lambda - lambda_old) > 0.0001 && iter < 1000);
  printf("Dominant Eigenvalue: %.4f\n", lambda);
}

void simultaneous_equations() {
  float a[Max][Max + 1]; int n;
  printf("Enter number of variables: ");
  if (scanf("%d", &n) != 1)
    return;
  printf("Enter augmented matrix: ");
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n + 1; j++)
      if (scanf("%f", &a[i][j]) != 1)
        return;
  for (int i = 0; i < n; i++) {
    if (a[i][i] == 0.0)
      return;
    for (int j = 0; j < n; j++) {
      if (i != j) {
```

```
        float ratio = a[j][i] / a[i][i];
        for (int k = 0; k < n + 1; k++) a[j][k] = a[j][k] - ratio * a[i][k];
      }
    }
  }
  printf("Solutions:\n");
  for (int i = 0; i < n; i++) printf("x%d = %.4f\n", i + 1, a[i][n] / a[i][i]);
}

void orthogonal() {
  float a[Max][Max], mul[Max][Max] = { 0 }; int n, isOrtho = 1;
  printf("Enter the matrix\n");
  input_matrix(a, &n, &n, 1);
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      for (int k = 0; k < n; k++) mul[i][j] += a[i][k] * a[j][k];
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      if (i == j && (mul[i][j] < 0.999 || mul[i][j] > 1.001)) isOrtho = 0;
      if (i != j && (mul[i][j] < -0.001 || mul[i][j] > 0.001)) isOrtho = 0;
    }
  }
  printf(isOrtho ? "Matrix is Orthogonal.\n" : "Matrix is NOT Orthogonal.\n");
}

int main() {
  int choice;
  do {
    printf("\n==== Matrix Operations Combined ====\n");
    printf("1. Addition\n2. Subtraction\n3. Multiplication\n4. Inverse\n5.
    Adjoint\n6. Determinant\n7. Eigenvalues\n8. Simultaneous Equations\n9.
    Orthogonal Check\n0. Exit\nEnter your choice: ");
    if (scanf("%d", &choice) != 1) break;
    if (choice == 1) addition();
    else if (choice == 2) subtraction();
    else if (choice == 3) multiplication();
    else if (choice == 4) inverse();
    else if (choice == 5) adjoint_op();
    else if (choice == 6) determinant();
    else if (choice == 7) eigenvalue();
    else if (choice == 8) simultaneous_equations();
    else if (choice == 9) orthogonal();
    else if (choice == 0) printf("Exiting...\n");
    else printf("Invalid choice!\n");
  } while (choice != 0);
  return 0;
}
```

Output:

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 1
Enter rows and columns: 2 2
Enter first matrix
Enter elements: 1 2 3 4
Enter second matrix
Enter elements: 3 4 5 6
Sum:
4.00 6.00
8.00 10.00
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 2
Enter rows and columns: 2 2
Enter first matrix
Enter elements: 1 2 3 4
Enter second matrix
Enter elements: 3 4 5 6
Difference:
-2.00 -2.00
-2.00 -2.00
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 3
Enter first matrix
Enter rows and columns: 2 3
Enter elements: 1 2 3 4 5 6
Enter second matrix
Enter rows and columns: 3 2
Enter elements: 7 8 9 10 11 12
Product:
58.00 64.00
139.00 154.00
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 4
Enter the matrix
Enter rows and columns: 2
Enter elements: 4 5 6 7
Inverse:
-3.5000 2.5000
3.0000 -2.0000
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 5
Enter the matrix
Enter rows and columns: 2
Enter elements: 1 2 3 4
Adjoint:
4.0000 -2.0000
-3.0000 1.0000
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 6
Enter the matrix
Enter rows and columns: 3
Enter elements: 1 2 3 4 5 6 7 8 9
Determinant = 0.0000
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 7
Enter the matrix
Enter rows and columns: 2
Enter elements: 1 2 3 4
Enter initial guess vector: -1 1
Dominant Eigenvalue: 5.3723
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 8
Enter number of variables: 2
Enter augmented matrix: 2 1 5 1 -1 1
Solutions:
x1 = 2.0000
x2 = 1.0000
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 9
Enter the matrix
Enter rows and columns: 2
Enter elements: 1 0 0 1
Matrix is Orthogonal.
```

```
==== Matrix Operations Combined ====
1. Addition
2. Subtraction
3. Multiplication
4. Inverse
5. Adjoint
6. Determinant
7. Eigenvalues
8. Simultaneous Equations
9. Orthogonal Check
0. Exit
Enter your choice: 0
Exiting...
```

Practical : 6

Lagrange’s Interpolation

Aim: Program to implement Langrange’s Interpolation formula.

Code:

```
#include <stdio.h>

#define MAXN 10

float cx[MAXN], cy[MAXN];

float f1(float x, float pointx[], int i, int n) {
  float numer = 1.0, deno = 1.0;
  for (int j = 0; j < n; j++) {
    if (j != i) {
      numer = numer * (x - pointx[j]);
      deno = deno * (pointx[i] - pointx[j]);
    }
  }
  return numer / deno;
}

float f2(float x, float pointx[], float pointy[], int n) {
  float sume = 0.0;
  for (int i = 0; i < n; i++) {
    sume = sume + f1(x, pointx, i, n) * pointy[i];
  }
  return sume;
}

int main() {
  int n;
  float x, result;
  printf("Enter number of data points: ");
  scanf("%d", &n);
  if (n <= 0 || n > MAXN) {
    printf("Invalid number of points.\n");
    return 1;
  }
  printf("Enter x and y values:\n");
  for (int i = 0; i < n; i++) {
    printf("x%d y%d: ", i, i);
    scanf("%f %f", &cx[i], &cy[i]);
  }

  printf("Enter the x value to interpolate: ");
  scanf("%f", &x);
  result = f2(x, cx, cy, n);
  printf("Interpolated value = %.4f\n", result);
```

```
  return 0;
}
```

Output:

```
Enter number of data points: 10
Enter x and y values:
x0 y0: 1 2
x1 y1: 2 3
x2 y2: 3 4
x3 y3: 4 5
x4 y4: 5 6
x5 y5: 6 7
x6 y6: 7 8
x7 y7: 8 9
x8 y8: 9 10
x9 y9: 10 11
Enter the x value to interpolate: 20
Interpolated value = 16.0000
```